AI Lab assignment 1

Tejas Patil
U24AI061

Q1->

```
#include<bits/stdc++.h>
using namespace std;
int cnt=1;
int cnt1=1;

vector<string> city = {
    "Chicago","Indianapolis","Columbus","Cleveland","Detroit",
    "Buffalo","Pittsburgh","Baltimore","Philadelphia",
    "New York","Boston","Providence","Syracuse","Portland"
};

const int INF = 1e9;
int n = 14;
vector<vector<int>> dist(n,vector<int>(n,INF));
vector<vector<pair<int,int>>> adj(n);


void BFS(int src,int dest){
    queue<pair<vector<int>,int>> q;
    q.push({{src},0});

    while(!q.empty()){
        auto [path,cost]=q.front();
        int last=path.back();
        q.pop();
        if(last==dest){
            cout<<cnt++<<"=>";
            for(int id:path){
                cout<< city[id]<<"->";
            }
            cout<< cost<<endl;
            continue;
        }
```

```
        for(auto [next,wt]:adj[last]){
            if(find(path.begin(),path.end(),next)==path.end()){
                auto newpath=path;
                newpath.push_back(next);
                q.push({newpath,wt+cost});
            }
        }
    }

}

void DFS(int curr,int dest, vector<int>&path,int
cost,vector<bool>&visited){
    if(curr==dest){
        cout<<cnt1++<<"=>";
        for(int id:path){
            cout<< city[id]<<"->";
        }
        cout<< cost<<endl;
        return ;
    }
    visited[curr]=true;

    for(auto[next,wt]:adj[curr]){
        if(!visited[next]){
            path.push_back(next);
            DFS(next,dest,path,cost,visited);
            path.pop_back();
        }

    }
    visited[curr]=false;
}
int main() {
    for(int i = 0; i < n; i++)
        dist[i][i] = 0;
    dist[0][4] = dist[4][0] = 283;
    dist[0][3] = dist[3][0] = 345;
    dist[0][1] = dist[1][0] = 182;
    dist[1][2] = dist[2][1] = 176;
```

```
    dist[2][3] = dist[3][2] = 144;
    dist[2][6] = dist[6][2] = 185;
    dist[3][4] = dist[4][3] = 169;
    dist[3][6] = dist[6][3] = 134;
    dist[3][5] = dist[5][3] = 189;
    dist[4][5] = dist[5][4] = 256;
    dist[5][12] = dist[12][5] = 150;
    dist[5][6] = dist[6][5] = 215;
    dist[6][8] = dist[8][6] = 305;
    dist[6][7] = dist[7][6] = 247;
    dist[7][8] = dist[8][7] = 101;
    dist[8][9] = dist[9][8] = 97;
    dist[8][12] = dist[12][8] = 253;
    dist[9][10] = dist[10][9] = 215;
    dist[9][11] = dist[11][9] = 181;
    dist[10][11] = dist[11][10] = 50;
    dist[10][13] = dist[13][10] = 107;
    dist[12][10] = dist[10][12] = 312;
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            if(dist[i][j]!=INF && i!=j){
                adj[i].push_back({j,dist[i][j]});
            }
        }
    }
    int src = 12; // Syracuse
    int dest = 0; // Chicago
    // BFS(src,dest);

    cout << "\nDFS Paths from Syracuse to Chicago:\n";
    vector<bool> visited(n, false);
    vector<int> path = {src};
    DFS(src, dest, path, 0, visited);
    return 0;
}
```

Q2->

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long int

unordered_map<string,vector<string>> graph;

void addFriend(string a,string b){
    graph[a].push_back(b);
    graph[b].push_back(a);
}

void BFS(string start){
    unordered_set<string> visited;
    queue<string> q;

    q.push(start);
    visited.insert(start);
    while(!q.empty()){
        string curr=q.front();
        q.pop();
        cout<< curr << endl;
        for(auto fri :graph[curr]){
            if(!visited.count(fri)){
                visited.insert(fri);
                q.push(fri);
            }
        }
    }
}

void DFS(string current,unordered_set<string>&visited){
    visited.insert(current);
    cout<< current<<endl;

    for(string near:graph[current]){
        if(!visited.count(near)){
            DFS(near,visited);
        }
    }
```

```
}
int main(){
    addFriend("Raj", "Priya");
    addFriend("Raj", "Sunil");

    addFriend("Priya", "Aarav");
    addFriend("Priya", "Akash");
    addFriend("Priya", "Neha");

    addFriend("Sunil", "Akash");
    addFriend("Sunil", "Sneha");

    addFriend("Akash", "Neha");

    addFriend("Neha", "Rahul");
    addFriend("Aarav", "Neha");

    addFriend("Sneha", "Rahul");
    addFriend("Sneha", "Maya");

    addFriend("Maya", "Arjun");

    addFriend("Rahul", "Arjun");
    addFriend("Rahul", "Pooja");

    addFriend("Arjun", "Pooja");

    cout<< "Travel using BFS";
    BFS("Sneha");

    cout<< endl<<endl;
    unordered_set<string>v;
    cout<< "Travel using DFS";
    DFS("Sneha",v);

    return 0;
}
```